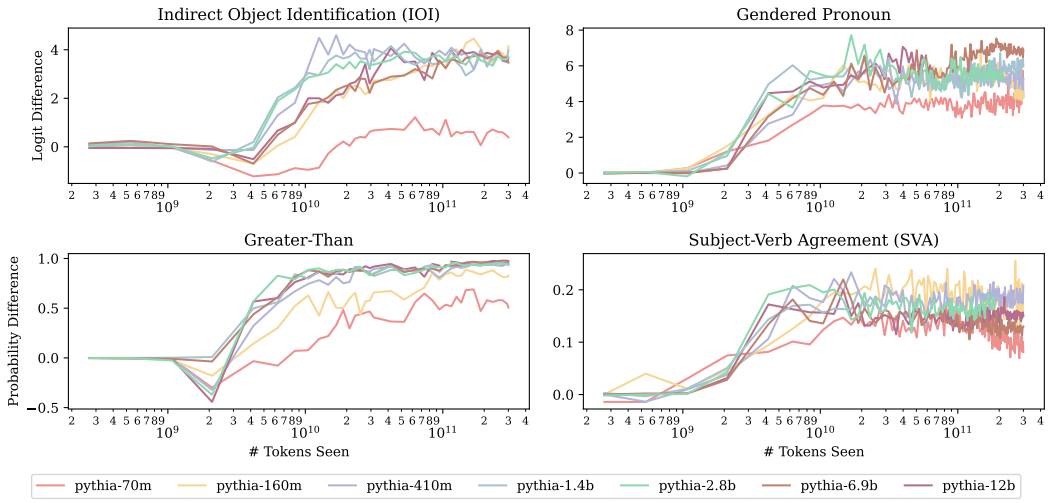

Figure 1: Task behavior across models and time (higher indicates a better match with expected behavior). Across tasks and scales, model abilities tend to develop at the same number of tokens.

Gendered-Pronoun The Gendered-Pronoun task [69, 44, 15] measures the gender of the pronouns that models produce to refer to a previously mentioned entity. Prior work has shown "So Paul is such a good cook, isn't"", models prefer the continuation "he" to "she"; we measure the degree to which this occurs via the difference in the pronouns' logits. In the corrupted case, we replace the "Jack" with "Mary"; we include opposite-bias examples as well. We craft 70 examples as in [44].

Greater-Than The Greater-Than task [32] measures a model's ability to complete inputs such as $s =$"The war lasted from the year 1732 to the year 17" with a valid year (i.e. a year > 32). Task performance is measured via probability difference (prob diff); in this example, the prob diff is $\sum_{y=33}^{99} p(y|s) - \sum_{y=00}^{32} p(y|s)$. In corrupted inputs, the last two digits of the start year are replaced by "01", pushing the model to output early (invalid) years that decrease the prob diff. We create 200 Greater-Than examples with Hanna et al.'s [32] generator.

Subject-Verb Agreement Subject-verb agreement (SVA), widely studied within the NLP interpretability literature [41, 52, 39], tasks models with predicting verb forms that match a sentence's subject. Given input such as "The keys on the cabinet", models must predict "are" over "is"; a corrupted input, "The key on the cabinet" pushes models toward the opposite response. We measure model performance using prob diff, taking the difference of probability assigned to verbs that agree with the subject, and those that do not. We use 200 synthetic SVA example sentences from [52].

Note that two tasks (IOI and Gendered-Pronoun) use logit difference as their metric, while the other two (Greater-Than and SVA) use probability difference. In general, we prefer to compute differences of logits rather than probabilities, as the former may be better at detecting important negative components when performing patching [75]. However, Greater-Than and SVA have multiple clean and corrupted answers, while logit difference, being unnormalized, is only compatible with one clean and corrupted answer. We thus compute a difference in probabilities instead, as in prior work.

3 Circuit Formation

3.1 Behavioral Evaluation

We begin our analysis of LLMs' task mechanisms over time by analyzing LLM behavior on these tasks; without understanding their task behaviors, we cannot understand their task mechanisms. We test these by running each model (Section 2.3) on each task (Section 2.4). Our results (Figure 1) display three trends across all tasks. First, all models but the weakest (Pythia-70m) tend to arrive at the same task performance at the end of training. This is consistent with our choice of tasks: they are simple, learnable even by small models, and scaling does not significantly improve performance.

Second, once models begin learning a task, their overall performance is generally non-decreasing, though there are minor fluctuations; Pythia-2.8b's logit difference on Gendered Pronouns dips slightly after it learns the task. In general, though, models tend not to undergo significant unlearning. The only marked downward trend (Pythia-70m at the end of SVA) comes from a weak model.

Finally, for each task we examined, we observed that there was a model size beyond which additional scale did not improve the rate of learning, and sometimes even decreased it; task acquisition appeared to approach an asymptote. We found this surprising due to the existence of findings showing the opposite trend for some tasks: [37, 57]. On some tasks (Gendered Pronouns and Greater-Than), all models above a certain size (70M parameters for Gendered Pronouns and 160M for Greater-Than) learn tasks at roughly the same rate. On IOI, models from 410M to 2.8B parameters learn the task the fastest, but larger models (6.9B and 12B) have learning curves more like Pythia-160m. We obtain similar results on more difficult tasks like SciQ [73]; for these results, see Appendix E.

What drives this last trend, limiting how fast even large models learn tasks? To understand this, we must delve into the internal model components that support these behaviors and trends.

3.2 Component Emergence

Prior work [54, 14, 63] has shown how a model's ability to perform a specific task can hinge on the development of certain components, i.e. the emergence of attention heads or MLPs with specific, task-beneficial behaviors. Prior work has also thoroughly characterized the components underlying model abilities two of our tasks, IOI and Greater-Than, at the end of training. We thus ask: is it the development of these components that causes the task learning trends we saw before? We focus on four main components, all of which are attention heads,[3] which we briefly describe here:

Induction Heads [54] activate on sequences of the form `[A] [B] . . . [A]`, attending to and upweighting `[B]`. This allow models to recreate patterns in their input, and supports IOI and Greater-Than.

Successor Heads [29] identify sequential values in the input (e.g. "11" or "Thursday") and upweight their successor (e.g. "12" or "Friday"); this supports Greater-Than behavior.

Copy Suppression Heads [45] attend to previous words in the input, lowering the output probability of repeated tokens that are highly predicted in the residual stream input to the head. In the original IOI circuit, copy suppression heads hurt performance, downweighting the correct name. In contrast, we find (Appendix D) that they contribute positively to the Pythia IOI circuit by downweighting the incorrect name; this is possible because both names are already highly predicted in the input to these heads, and they respond by downweighting the most repeated one.

Name-Mover Heads [71] perform the last step of the IOI task, by attending to and copying the correct name. Unlike the other heads described so far, this behavior is specific to IOI-type tasks; their behavior across the entire data distribution has not yet been characterized.

Because the importance of these components to IOI and Greater-Than has been established in other models, but not in necessarily in those of the Pythia suite, we must first confirm their importance in these models. To do so, we find circuits for each model at each checkpoint using EAP-IG, as described in Section 2.2; note that circuits found at early checkpoints, where task performance is poor, are generally not meaningful. We omit Pythia-6.9b and 12b from circuit finding for reasons of computational cost. We then verify via path patching [28] that these component types appear within Pythia models' tasks circuits; see Appendices A and B for details on our methods and findings.

For each component, prior work has developed a metric to determine whether a model's attention head is acting like that component type; see Appendix D for details on these. Using these metrics, we score each of the heads in our models' circuits at each checkpoint, evaluating the degree to which each in-circuit head exhibits the aforementioned component behaviors. We then sum the score for each component across all in-circuit heads at each checkpoint, measuring how strongly the component behavior exists overall in the circuit at that checkpoint. We then normalize this sum across checkpoints, to understand how this behavior develops over time.

Our results (Figure 2) indicate that many of the hypothesized responsible components do emerge the same time as model performance increases. Most models' induction heads emerge soon after

[3]Though past work on e.g. Greater-Than studied task-relevant MLPs and neurons, what generalizable behavior they have is poorly understood, unlike the attention heads we study.

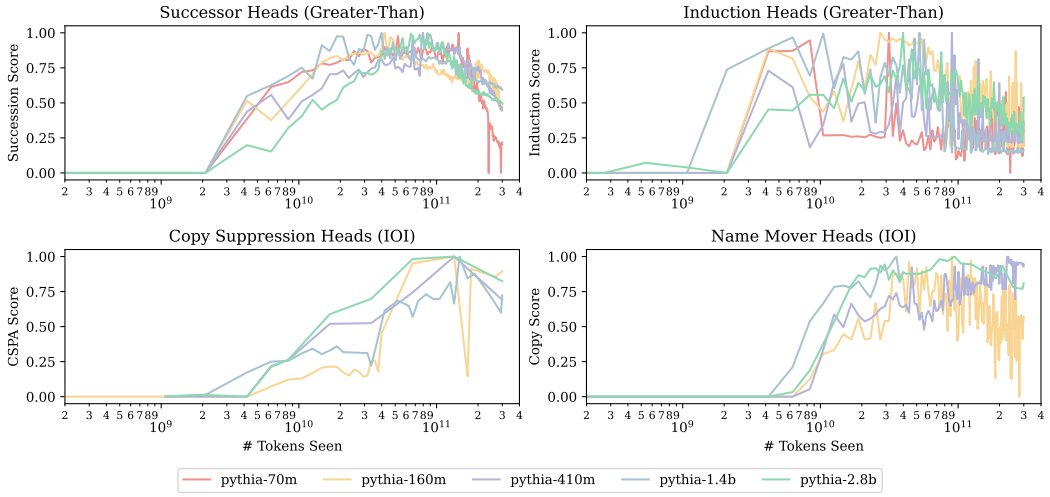

Figure 2: The development of components relevant to IOI and Greater-Than, across models and time. Each line indicates the degree to which attention heads in the circuit at each timestep exhibit the relevant component behavior. The timesteps at which component behavior emerges parallel those at which task performance emerges in Figure 1.

they have seen 2×10^9 tokens, replicating the findings in [54]; immediately after this, Greater-Than behavior emerges. The successor heads, also involved in Greater-Than, emerge in a similar timeframe. Curiously, the behavior of these heads eventually decreases, while task performance does not. We note that this could happen if the number of heads exhibiting successor behavior decreases, while the impact of said heads (on the model's logits) increases.

For IOI, the name-mover heads emerge at similar timesteps (2 - 8×10^9 tokens) across models, with a very high strength, during or just before IOI behavior appears. Copy suppression heads emerge at the same timescale, but at varying speeds, and with varying strengths. Given that these heads are the main contributors to model performance in each task's circuit, and they emerge as or just before models' task performance increases, we can be reasonably sure that they are responsible for the emergence of performance. The apparent ceiling on the speed at which these heads emerge, even for larger models, may be responsible for the same ceiling on how fast model performance emerges, underlain by these heads.

Throughout this section's analyses, we note an unusual trend: though model performance (Figure 1) does not decrease over time, the functional behavior of certain attention heads does. In the following section, we explore how this occurs.

4 Algorithmic Stability and Generalizability in Post-Formation Circuits

We demonstrated in Section 3 that across a variety of tasks, models with differing sizes learn to perform the given task after the same amount of training; this appears to happen because each task relies on a set of components which develop after a similar count of training tokens across models. However, in Figure 2, we observed that attention heads that performed a given functional behavior at a given point in training may later perform that behavior more weakly or not at all. This raises questions: when the heads being used to solve a task change, does the algorithm implemented by the model change too? And how do these algorithms generalize across model scale?

4.1 Model Behavior and Circuit Components Post-Formation

To understand how model components change over time, we analyze components and their algorithmic roles across training. We now zoom in on the components in one model, Pythia-160m, and study them over the course of training; where we earlier plotted only the top component (e.g. the top successor head), of each model, we now plot the top 5 of Pythia-160m's heads that exhibit a given functional behavior (or fewer, if fewer than 5 exist). By evaluating components and algorithms over